

VRES Project Documentation - 3D agent-based model of T cell expansion in micro rod scaffolds

1. Context (taken from abstract)

- T cells form part of the body's immune system and can recognise and kill cancer cells, but often there aren't enough of them to fully eliminate a tumour.
- Adoptive T cell therapy addresses this by extracting T cells, expanding (multiplying) them outside the body and reinfusing them.
- One promising way to expand T cells ex vivo is using micro-rod scaffolds.
- These rod-like structures mimic aspects of the immune environment and help trigger T cell activation and proliferation.
- However, there are currently no 3D mathematical models that realistically capture both micro-rod movement and their mechanical interactions with T cells.

This codebase implements a 3D agent-based model simulating T cell activation and proliferation within microrod scaffolds within Physicell. The model captures:

- Physical interaction between T cells and micro rods which trigger activation
- IL-2 secreted by the rods driving T cell proliferation
- Mechanical forces keeping the rods semi-rigid and movable when T cells push against them

2. Cell Types

In the 3D model, 2 cell types are defined in a small 3d environment:

Cell Type	Key Behaviours
T cell	• Motile spherical agent. Can be active or naïve. • Moves through domain with bias towards chemokine • Activates after a certain amount of physical contact (30mins) with rods. • Once active, they are able to proliferate based on the local IL-2 concentration • Daughter cells are reset to naïve after each division and must be activated independently
Rods	• Composed of rod cells. • Does not move in its own (non-motile) but can be pushed by T cells • Secretes IL-2 at a time-decaying rate. • Is built from many spherical rod agents held together by elastic spring attachments • Maintains semi rigid structure based on specific structure and arrangement of attachments

A significant part of the project was spent on how rods are built and how they behave mechanically.

3. Why rod construction is non-trivial in PhysiCell

Microrods are long, semi-rigid structures but PhysiCell represents every cell as a sphere. Therefore, a simple first approach was to model each microrod as a chain of connected spherical rod cells with attachments between neighbouring cells.

3.1 PhysiCell Attachments

The core PhysiCell code specifies a function called `rod_elastic_contact_with_confluent_length` which is used to attach two cells to each other.

At a high level, the attachment function connects neighbouring spherical agents using elastic spring like forces. These forces continuously try to keep the spheres at specific distances from one another.

If two attached spheres drift too far apart, the attachment force pulls them back together. If they become too compressed, the force pushes them apart. By applying these restoring forces, two cells are effectively attached to each other.

In both the 2D and 3D solution, a modified version of this function is used to attach rod cells together to form the rod structure.

Now if we were to string together a single line rod cells each attached to the neighbouring cell, in 2D the following tends to happen:

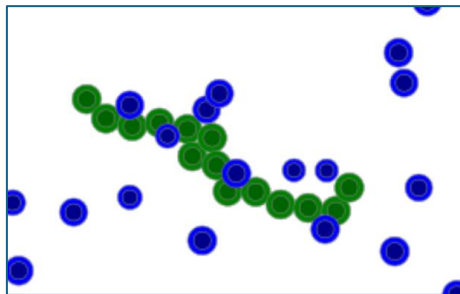


Figure 1 - 2D rod cells attached in one line (bending and shearing observed)

While each sphere is attached to the one next to it, they move independently causing the rod to bend and shear easily.

One way to prevent this is to make the rod agents immovable. However, in actual microrod experiments, T cells physically push and cluster the rods and this clustering is thought to drive T cell expansion. Since understanding this phenomenon is a core goal of the model, we require rods that are both rigid and movable, they should resist bending excessively but still respond to forces applied by T cells.

The main challenge therefore becomes: how do we make a collection of spherical agents behave as a semi-rigid object?

3.2 The solution in 2D

The solution that worked in 2D was to have two parallel rows of spherical agents, staggered by half a cell width, form each rod. Attachments along each row, diagonal bonds between the rows, and longer-range bracing bonds spanning multiple cells along the rod axis together provide rigidity as seen in Figure 2.

Note that in the 2D code, the variable k specifies for the longer-range attachments how many cells ahead in the same row each cell gets attached to. So, with $k = 4$, cell 0 attaches to cells 1, 2, 3, 4. Cell 1 bonds to cells 2, 3, 4, 5. And so on.

I believe Brynn used $k=9$ for her presentation but this can be changed in the code.

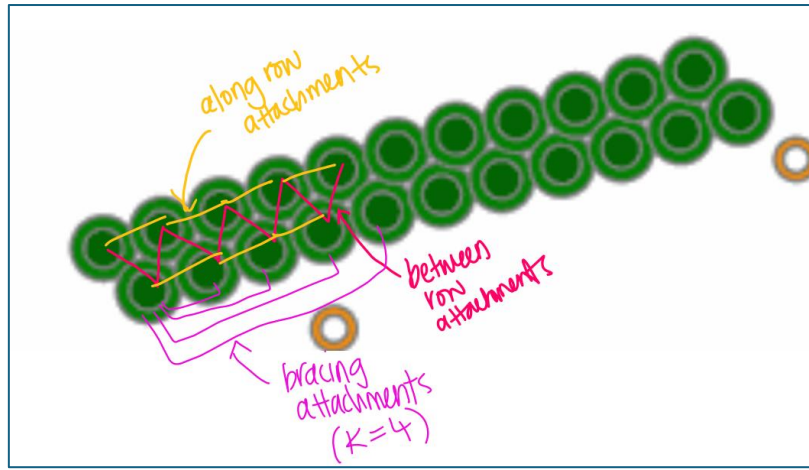


Figure 2 - 2D semi-rigid rod structure

While the solution worked when rods were not overlapping, when multiple rods were placed randomly in the domain, rod agents could overlap in 2D (especially at higher rod concentrations). When this occurred, multiple rod cells would occupy the same position in the domain at the same time causing numerical instability and as a result, rods and attachments would fall apart as seen in Figure 3.

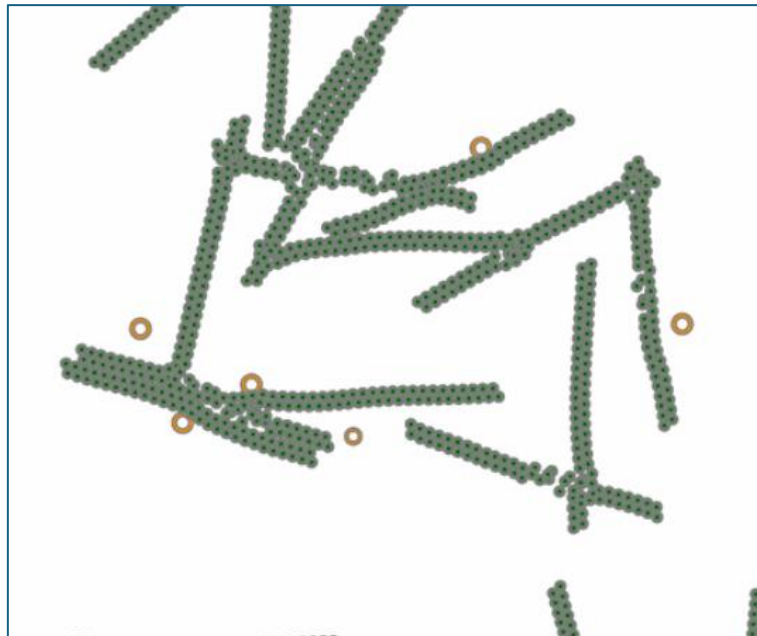


Figure 3 - Rod overlapping in 2D causing instability

This prompted the switch to 3D since we could separate rods onto distinct z-planes and there would be no overlap in space.

3.3 The solution in 3D

3.3.1 Initial approach

The most natural extension from the 2D solution into 3D was to add a third row positioned in the gaps between the top and bottom rows creating a triangular cross section. Attachments were then added from this middle row to both the top and bottom rows to form a 3D rod structure (Figure 4).

Several interesting behaviours were observed during the simulations using this structure. When no longer-range bracing attachments were added, the rod generally maintained its initial shape but was prone to excessive bending and shearing (Figure 5). However, when a large number of bracing

attachments were introduced (for example, $k=9$), the rod developed a strange unstable shape instead (Figure 6).

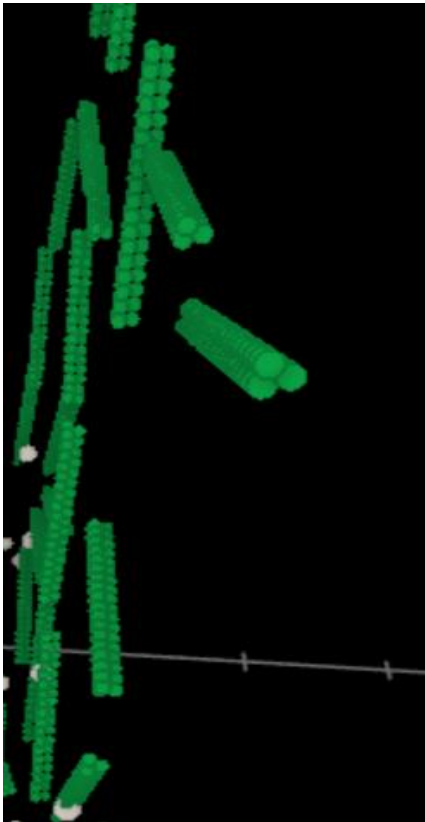


Figure 4 - Initial 3D rod set up (extension from 2D)

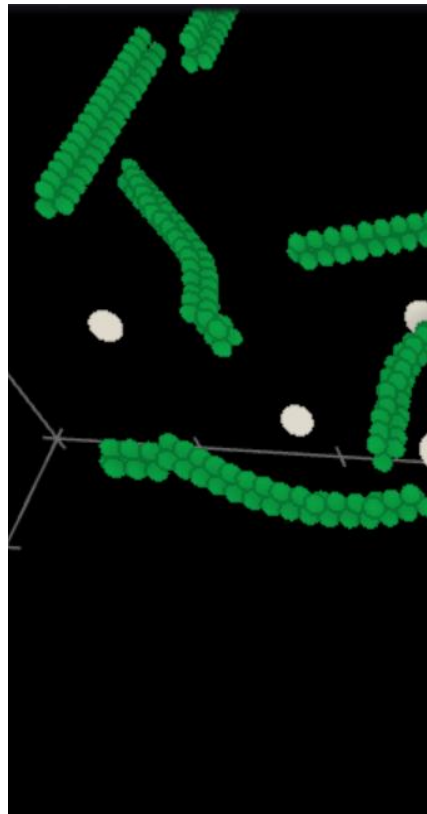


Figure 5 - Initial 3D rod set up $K=1$, no longer range bracing

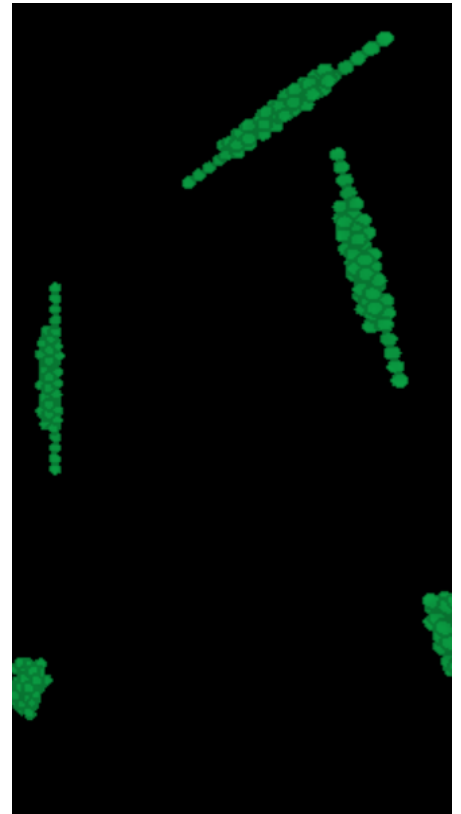


Figure 6 - Initial 3D rod set up with longer range bracing $K=9$

I'm still not entirely sure why this occurred. One possibility is that it was related to how the rest lengths for the longer bracing attachments were defined during initialisation. Even small inaccuracies may have caused the rod cells to rearrange themselves into a different but mechanically stable configuration (???) Another possibility is that extending the model from 2D into 3D introduced additional degrees of freedom and instability which then affected the final rod shape. Or it may be something else entirely. However, at $K=2$, an observation I made was that, at times, the rod would rearrange into a stable shape such the ones circled in red in Figure 7:

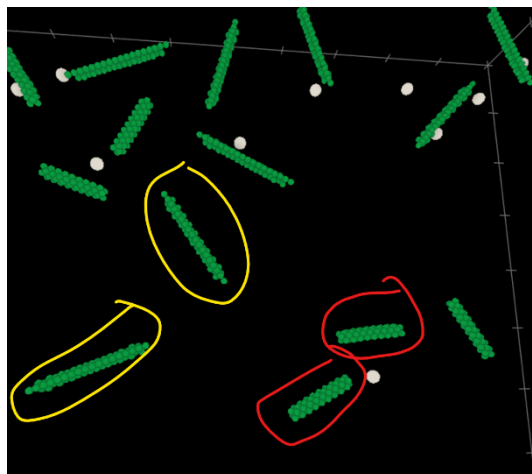


Figure 7 - Initial 3D rod set up with longer range bracing $K=2$

This stable shape appeared to be rigid enough while still remaining movable, making it ideal in terms of micro rod structure and mechanics. However, with the initial 3D set up, it did not consistently form. Rod would sometimes also settle into a more elongated shape as seen in the rods circled in yellow. Therefore, I changed the initial rod configuration to better mimic this structure of the rods in red so that the rods were more likely to mechanically settle into it.

3.3.2 Final 3D rod solution Structure

The stable structure that formed somewhat resembled hexagonal close packing (HCP), a common 3D crystal structure (Figure 8).

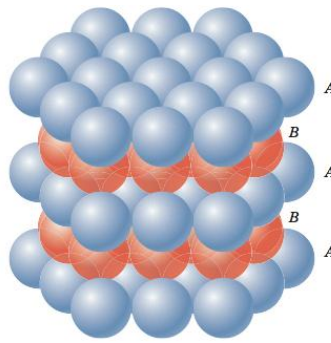


Figure 8 - Hexagonal Close Packing Structure

HCP is characterised by layers stacked in an alternating ABAB pattern, where the first and third layers align, and the second and fourth layers also align.

To reduce computational cost, a simplified HCP-inspired triangular structure was used for rod construction. The final 3D rod design seen in Figure 9 (which is described in more detail in the code section) includes the following features:

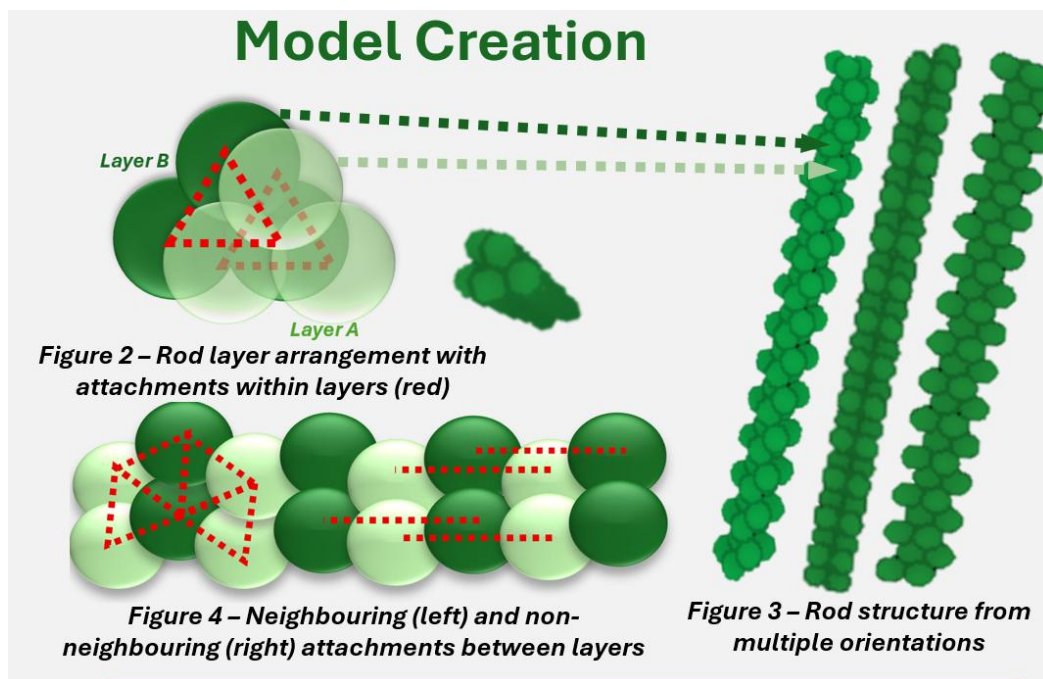


Figure 9 - Rod Structure inspired by HCP

ABAB-stacked triangular cross-section

Each rod layer contains three agents arranged in an equilateral triangle. Consecutive layers are offset so that each new layer sits in the gaps of the previous one, following an HCP-like arrangement.

Longitudinal bracing attachments

Connections within a layer and between adjacent layers are not enough to fully prevent shearing along the rod. To improve stability, additional “bracing” attachments are added between layers that are two steps apart.

Each attachment uses its initial spacing as a rest length, meaning the elastic forces always try to restore the rod to its original shape rather than letting it compress into a uniform structure.

4. Code overview

The code structure is as follows:

Function	What it does
Helper functions	Vec3 maths Cell attachment utilities 2D geometry checks for rod placement
create_cell_types()	Specifies T cell and rod definitions Assigns function pointers
setup_microenvironment()	Initialises the diffusion grid for IL-2 and chemokine
setup_tissue()	Places all T cells and rods in the domain at simulation start
contact_function()	Applies elastic spring forces between attached rod cells
tcell_update_velocity()	Moves T cells Detects rod contact Manages activation logic
tcell_division_function()	Resets daughter cells to naive after division
cell_proliferation_based_on_IL2()	Sets T cell division rate from local IL-2 level
secretion_rate_rod_cells()	Sets the IL-2 secretion rate for rod cells at each time step
coloring_function()	Controls cell colours in the PhysiCell SVG output
compute_number_of_rods_from_concentration()	Converts rod concentration ($\mu\text{g/mL}$) to a rod count
clamp_cell_to_domain()	Prevents cells from being placed outside the domain boundaries

5. Rod Construction Simplified

Rod construction and placement occur primarily in `setup_tissue()`

The following algorithm describes how rods are constructed and placed.

Step 1: Decide how many rods

Function: `compute_number_of_rods_from_concentration()`

What it does: Based on the concentration of rods ($\mu\text{g/mL}$) provided in the user parameters (`rod_concentration_ug_per_mL`) in the xml file:

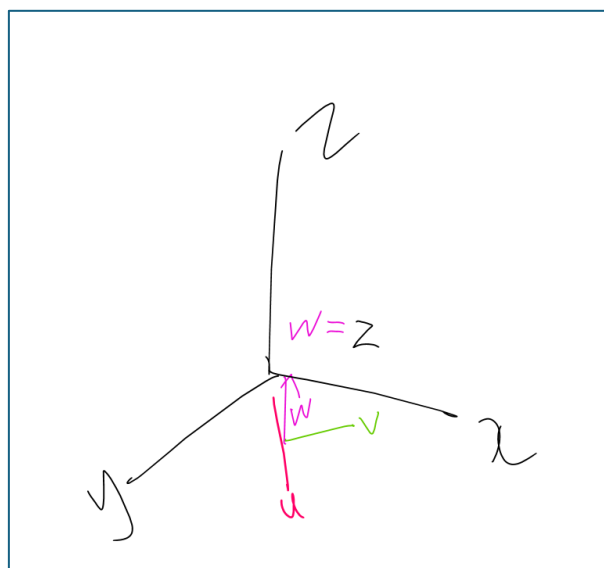
- Takes rod concentration ($\mu\text{g/mL}$)
- Converts total simulation volume to mL
- Estimates total rod mass based on total simulation volume
- Divides by mass per rod
- This outputs the total number of rods required to have that concentration of microrods.

Step 2: Choose random rod position and orientation

For each rod:

- Assign random (x, y) position and
- random angle which defines the orientation of the rods.

Both of these aspects help to define the vector u , which runs along the rod. We also compute perpendicular directions v and w (where w is same as z or straight up) to build the structure around the rod. See diagram below.



Step 3: Build the geometry (Hexagonal Close Packing layers)

Using the HCP inspired structure, each rod is built from layers:

- Each layer = 3 cells forming a triangle
- Layers stack along direction u
- Every second layer is shifted (ABAB pattern)

This was chosen because it was found that this pattern allows for mechanical stability in 3D.

Step 4: Place cells in space

For each layer:

- Move along the rod axis (u)
- Add triangle offsets (in $v - w$ plane)



Step 5: Add attachments between the rod cells

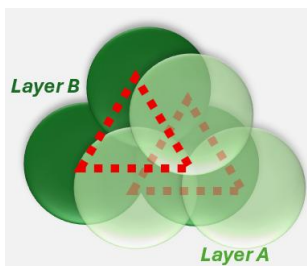
Function: `attach_cells_with_rest()`

Each connection:

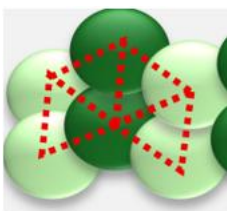
- Links two cells
- Stores their initial distance (rest length)

Types of attachments:

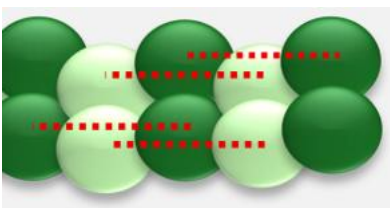
- Within a layer (triangle edges)



- Between layers (nearest neighbours)



- Across layers (longitudinal braces)



Step 6: Store rest lengths

Function: `get_rest_length()`

Each attachment stores the actual distance between the two agents at the moment it was created. This is called the rest length. The elastic spring force always tries to restore this original spacing, so the rod resists both compression and stretching while still responding to external forces from T cells.

Step 7: Rod Mechanics (How forces are applied)

Handled by:

Function: `contact_function()` and `rod_elastic_contact_thin_hcp()`

`rod_elastic_contact_thin_hcp()` is based off the in-built PhysiCell function `rod_elastic_contact_with_confluent_length()`

The contact function applies the `rod_elastic_contact_thin_hcp()` to all attached cells. At each time step, when each pair of attached cells move, the function:

- Compute current distance
- Compare to rest length
- Apply force to restore original spacing

This gives us the behaviour that when two cells move too far apart, they pull together and if they are too close, they push apart. This enables rod to maintain their semi rigid structure.

Step 8. Rod Placement and avoiding overlap between rods

In PhysiCell when two cells occupy the same space at the same time, it produces unstable behaviour. When simulating in 2D, this was a problem because for higher rod concentration, rod overlap was inevitable. This is what initially prompted the change from 2D to 3D. In 3D, this issue can be overcome by having multiple z layers.

So, at the start of the simulation, a set number of z layers are specified by the user parameter `num_rod_levels`. A rod can lay flat on any of the z layers. When randomly placing rods, a random z layer is initially assigned. Then we check if there is any crossing or overlap in that particular to know if the placement fails. Note that on each layer, the rod remains flat so we only have to check overlap for that specific layer.

To do so, in `setup_tissue()`, we check that:

- Rods don't cross (`segments_intersect2d()`)
- Rods aren't too close (`dist2_segment_to_segment()`)

If either is true, placement fails. In that case:

- Try a different z layer and continue the check again.

6. T Cell Behaviour

Movement and Contact

Function: `tcell_update_velocity()`

- Find nearby cells
 - Check if any are rods cells
- If distance between t cell and rod is small enough, it means the two are touching

Activation

Still in `tcell_update_velocity()`

Each T cell tracks three custom variables:

Variable	Values	Meaning
touching_rod	0 or 1	Whether the cell is in contact with a rod this time step
attached_time	minutes	Total time the cell has spent touching a rod
state	0 or 1	0 = naive (inactive), 1 = active

The activation logic is:

- Each time step, if the T cell is touching a rod, `attached_time` is incremented by `dt`
- Once `attached_time` reaches 30 minutes, `state` is set to 1 (active)
- Activation is permanent, the cell does not deactivate if it loses contact

Proliferation

Function: `cell_proliferation_based_on_IL2()`

- Only active cells divide
- Rate depends on IL-2

Higher IL-2 means faster division

Division Reset

Function: `tcell_division_function()`

By default, when a T cell proliferates, the daughter cell retains the characteristic of the parent cell. Since only active T cells proliferates, by default the daughter cells are active. This function is used to customise this such that once a t cell proliferates, the daughter cells reset too inactive (`state = 0`) and contact history (or `attached_time`) is set to 0.

7. IL-2 Model

IL-2 is secreted by rod cells and this is controlled via:

Function: `secretion_rate_rod_cells()`

- Rods release IL-2 over time
- Decays exponentially

Diffusion

Handled automatically by PhysiCell. Each T cell reads IL-2 at its location and T cell proliferation is based on this value.

8. Key XML Parameters

All of the following can be changed in the `PhysiCell_settings.xml` file without recompiling the code.

Parameter	Description
number_of_T_cells	Initial number of T cells
rod_concentration_ug_per_mL	Physical rod concentration used to compute rod count
rod_cells_per_rod	Number of spherical agents per rod
num_rod_levels	Number of Z-planes available for rod placement
rPmax	
IP	
v	
q	
rho	

8. Running the Code

For the 3D code, use the standard commands from the main directory:

```
make reset
```

```
make VRES-fresh
```

```
./VRES
```

For the 2D simulations, it is possible to create GIFs from the output images produced by PhysiCell using ImageMagic or even using python libraries that convert svgs to png and gifs.

For the 3D simulations, the output can be viewed in [Simularium](#) directly in your browser by dragging and dropping a .simularium file into the viewer. The .simularium file can be generated using simularium.py.

Set path_to_output_dir to the output directory containing all of the simulation files. _

9. Limitations and Future Work

A key limitation of the current 3D model is that clustering between T cells and microrods does not naturally occur. Note it seems in the 2D model, clustering was somewhat observed. Check with Brynn.

A next step could be to introduce and systematically tune adhesion between T cells and microrods as this could promote clustering.

Additional future improvements:

- Adjusting rod cell mass may allow rods to respond more realistically when pushed by T cells
- Increasing chemokine bias or persistence time could encourage T cells to move more directly toward rods and remain in their vicinity longer,
- Introducing changes in T cell motility when near rods (e.g., slowing down or increased interaction probability) which could help promote clustering.